Aspectos destacados

Concatenación Top-k de descriptores para clasificar texturas: comparación del uso directo y la activación por margen

Carlos Ayala, Joaquín Delgado

- Top-4 usa el 15 % de las dimensiones y queda cerca de la concatenación completa.
- La activación por margen conserva el macro-F1 de Top-k con 75,5 % menos costo.
- El margen elige qué imágenes escalar mejor que un sorteo en 27 de 27 condiciones.
- En un panel online, la política por margen tarda entre 22 y 54 ms por imagen.
- Las pruebas no detectan diferencias entre el uso directo y la activación por margen.

Concatenación Top-k de descriptores para clasificar texturas: comparación del uso directo y la activación por margen⋆

Carlos Ayala[a,*,1], Joaquín Delgado[a,1]

[a] *Universidad Nacional de Asunción, Facultad Politécnica (UNA-FP), Campus Universitario, San Lorenzo, Central, Paraguay*

ARTICLE INFO

Keywords:
clasificación de texturas
concatenación de descriptores
selección Top-k
activación por margen
clasificación en cascada
costo de inferencia

Abstract

La concatenación de descriptores heterogéneos mejora la clasificación de texturas, pero obliga a extraer todos los bloques en cada imagen. Este trabajo compara dos formas de usar una concatenación Top-k, formada por los k descriptores mejor posicionados en un ranking calculado solo con el entrenamiento: el uso directo, que la calcula para toda imagen, y la activación por margen, que primero clasifica con un descriptor barato y calcula Top-k solo cuando la diferencia entre las dos clases más puntuadas es menor que un umbral fijado con el entrenamiento. Sobre una biblioteca de 22 descriptores, seis datasets (DTD, FMD, CUReT, Outex, Soil y KTH-TIPS2-b) y dos clasificadores, SVM lineal y ResMLP, la curva de desempeño se aplana alrededor de $k = 4$. Top-4 usa cerca del 15 % de las 20.652 dimensiones y obtiene un macro-F1 medio de 0,9366 con SVM y 0,9331 con ResMLP, frente a 0,9391 y 0,9288 de la concatenación completa. En 27 condiciones con SVM, la activación por margen obtiene 0,9345 frente a 0,9341 de la misma Top-k usada siempre, con un costo proyectado 75,5 % menor; un piloto en GPU midió 90 frente a 291 ms por imagen. Con la misma cantidad de imágenes escaladas, el margen supera a un escalamiento aleatorio en las 27 condiciones. En un panel online de 94 condiciones, una política por margen con rescate de cuatro bloques tarda entre 22 y 54 ms por imagen, frente a 71–334 ms de Top-4, con un macro-F1 medio similar. Las pruebas no detectan diferencias entre ambos usos, lo que no equivale a demostrar equivalencia.

1. Introducción

La clasificación de texturas depende de señales locales, distribuciones espaciales y respuestas en frecuencia que cambian con el dominio, la escala, la iluminación y el punto de vista [1–3]. Los descriptores clásicos resumen propiedades diseñadas, como microestructuras, coocurrencias y gradientes [4–6]; las redes convolucionales, los Transformers y los modelos autosupervisados producen características aprendidas con arquitecturas y objetivos de preentrenamiento diferentes [7–11]. Ninguna de estas representaciones captura necesariamente todas las variaciones, y combinarlas es una estrategia habitual para mejorar el reconocimiento [2, 12, 13].

La forma más directa de combinarlas es concatenar los vectores de varios extractores y entrenar un único clasificador sobre el espacio conjunto. El costo de esa concatenación crece con cada bloque, porque para clasificar una imagen deben ejecutarse todos los extractores que la componen. En la biblioteca de 22 descriptores utilizada aquí, la concatenación completa suma 20.652 dimensiones y, en el panel online de este trabajo, tarda entre 1,55 y 1,82 segundos por imagen. Ese costo puede reducirse por dos vías complementarias: usar menos bloques o calcularlos en menos imágenes.

La primera vía conduce a una concatenación Top-k: se ordenan los descriptores por su desempeño individual en validación interna y se concatenan los k primeros. Su uso directo calcula esos k bloques para toda imagen. La segunda vía proviene de la inferencia condicional, en la que un modelo barato resuelve los casos sencillos y uno costoso se reserva para los difíciles [14–16]. Aplicada a Top-k, una etapa barata

⋆Borrador de trabajo de Tesis de Grado, Facultad Politécnica, Universidad Nacional de Asunción (UNA-FP). Documento preparado para revisión con el tutor.

*Autor de correspondencia

carlos.ayala@est.fpuna.edu.py (C. Ayala); joaquin.delgado@est.fpuna.edu.py (J. Delgado)

[1]Ambos autores contribuyeron al diseño y ejecución del estudio. Tutor: Prof. José Vázquez.

clasifica cada imagen con un único descriptor y la concatenación se calcula solo cuando el margen entre sus dos clases más puntuadas es pequeño. Esta activación por margen puede ahorrar extracción sin cambiar el modelo Top-k, pero solo funciona si el margen identifica las imágenes en las que la etapa barata se equivoca.

La pregunta de estudio es *cuánto desempeño se resigna al reemplazar la concatenación completa por una Top-k corta, y cuánto costo se ahorra al activarla solo cuando la etapa barata duda, frente a usarla siempre.* Para responderla se evaluaron 22 descriptores sobre DTD, FMD, CUReT, Outex, Soil y KTH-TIPS2-b con SVM lineal y ResMLP, bajo un protocolo anidado que realiza toda selección dentro del entrenamiento. Primero se traza la curva de desempeño en función de k y se fija $k = 4$; luego se compara Top-4 con el descriptor individual elegido internamente y con la concatenación completa en 47 condiciones externas por clasificador. La activación por margen se evalúa en 27 condiciones con SVM, frente a la misma Top-k usada siempre y frente a un escalamiento aleatorio con igual cantidad de imágenes. Por último, se miden latencias por imagen en un piloto físico y en un panel online de 94 condiciones.

El trabajo aporta tres resultados. Muestra que, con el ranking interno, la curva de macro-F1 se aplana alrededor de cuatro bloques, que ocupan cerca del 15 % de las dimensiones de la biblioteca. Compara de forma pareada el uso directo de Top-k y su activación por margen, con un control que separa el efecto del margen del efecto de escalar más imágenes. Y contrasta esas políticas con mediciones de latencia por imagen y con cuatro métodos recientes de concatenación evaluados sobre las mismas particiones. No se propone un nuevo extractor ni un nuevo algoritmo de selección: el objetivo es medir qué se gana y qué se pierde con cada forma de usar una concatenación corta.

El resto del artículo se organiza de la siguiente manera. La segunda sección revisa la concatenación de descriptores, la inferencia condicional y los métodos recientes de concatenación para texturas. La tercera describe la biblioteca, el ranking interno, la concatenación Top-k y la activación por margen. La cuarta presenta los datasets, los clasificadores, el protocolo anidado, el análisis estadístico y los resultados experimentales. La quinta discute los hallazgos y sus limitaciones. Finalmente, la sexta sección concluye.

2. Trabajos relacionados

2.1. Concatenación y selección de bloques

Los descriptores de textura resumen señales distintas de una misma imagen: microestructuras (LBP), coocurrencias, orientación de gradientes y patrones en frecuencia [4–6, 17]. Las representaciones aprendidas introducen sesgos adicionales derivados de arquitecturas convolucionales, Transformers y objetivos de preentrenamiento supervisado, autosupervisado o multimodal [7–11, 18–21]. Vectores de distinto origen pueden contener información redundante, pero también señales complementarias aprovechables por un clasificador conjunto.

La fusión por concatenación conserva las coordenadas de cada representación y permite entrenar un único clasificador sobre el espacio conjunto, sin introducir un módulo de fusión parametrizado. Los Fisher vectors [12], los bancos de filtros profundos [2] y DeepTEN [13] mostraron que reunir respuestas distintas puede mejorar la representación de texturas. El vector resultante, sin embargo, crece con cada extractor incorporado y puede acumular escalas incompatibles, variables redundantes y dimensiones poco informativas.

Elegir qué bloques concatenar es un problema de selección de características por grupos. Los métodos *wrapper* valoran subconjuntos con el propio clasificador y exploran combinaciones, por ejemplo mediante búsqueda secuencial hacia adelante; los filtros puntúan cada variable o bloque por separado y conservan los mejores [22, 23]. Top-k pertenece a esta segunda familia: evalúa cada descriptor una sola vez y no examina sus interacciones, lo que la hace barata y estable, aunque puede reunir bloques redundantes. En ambos casos, la selección debe realizarse dentro del entrenamiento y separarse de la evaluación externa [22, 23].

2.2. Inferencia condicional y costo de adquisición

Las cascadas reducen el costo medio de inferencia al aplicar primero clasificadores baratos y reservar los costosos para los casos que aquellos no resuelven. El detector de Viola y Jones descarta la mayoría de las ventanas con etapas simples y aplica etapas más complejas solo a las restantes [14]. En redes profundas, BranchyNet agrega salidas tempranas que terminan la inferencia cuando la predicción es suficientemente confiable [15], y MSDNet organiza clasificadores intermedios para operar bajo presupuestos de cómputo por

imagen o por lote [24]. Han et al. reúnen estas y otras redes dinámicas que adaptan su cómputo a cada entrada [16].

Otra línea trata cada característica como una medición con costo de adquisición. Greedy Miser incorpora ese costo al entrenamiento para clasificar bajo un presupuesto en test [25]; Nan y Saligrama aprenden una compuerta que decide, para cada entrada, si basta un modelo de bajo costo o debe consultarse uno de alto costo [26]; y EDDI elige de forma secuencial qué variable adquirir según la información esperada [27]. La activación por margen estudiada aquí es una versión simple de esa idea: los extractores congelados son las mediciones costosas, la confianza se mide con la diferencia entre las dos mayores puntuaciones de la etapa barata y el umbral se fija con márgenes fuera de pliegue del entrenamiento, sin entrenar una compuerta adicional.

2.3. Concatenación en texturas y métodos recientes

Varios trabajos recientes concatenan representaciones para clasificar texturas. Neshov et al. agregan capas internas y finales de arquitecturas preentrenadas, entre ellas BEiTv2, y entrenan una MLP sobre la concatenación; informan que el efecto de la agregación depende de la arquitectura [28]. Scabini et al. comparan 25 variantes Transformer como extractores congelados y encuentran que BEiTv2-B/16 obtiene el mejor promedio entre sus modelos [29]. DeepLumina combina características profundas con información de luminancia para texturas en color [30]; HyTexNet fusiona una codificación local guiada por percentiles con características de DenseNet-121 [31]; y Chang et al. concatenan LBP multiescala con contornos obtenidos por redes neuronales celulares profundas [32]. Estos métodos fijan de antemano qué representaciones se combinan y las calculan para toda imagen. El presente trabajo elige los bloques mediante un ranking interno sobre una biblioteca heterogénea y estudia en qué imágenes conviene calcularlos.

3. Método

3.1. Representación conjunta

Cada extractor transforma una imagen x_i en un vector $\mathbf{z}_{ij} \in \mathbb{R}^{d_j}$, donde j identifica el descriptor y d_j su dimensionalidad. Para un subconjunto S de extractores, la representación conjunta se define como

$$\mathbf{z}_{i,S} = \Big\Vert_{j \in S} \frac{\mathbf{z}_{ij}}{\max(\|\mathbf{z}_{ij}\|_2, 10^{-12})}, \tag{1}$$

donde $\Vert$ denota concatenación. La normalización L_2 se aplica por descriptor y por muestra antes de unir los bloques, para evitar que la escala de un extractor determine por sí sola su contribución al clasificador. La dimensión conjunta es $d_S = \sum_{j \in S} d_j$.

La Figura 1 muestra cómo una misma imagen produce representaciones diferentes. Un descriptor resume patrones, relaciones entre intensidades o activaciones aprendidas en un vector. Después de normalizar cada bloque, se conserva un descriptor individual o se concatenan los bloques seleccionados.

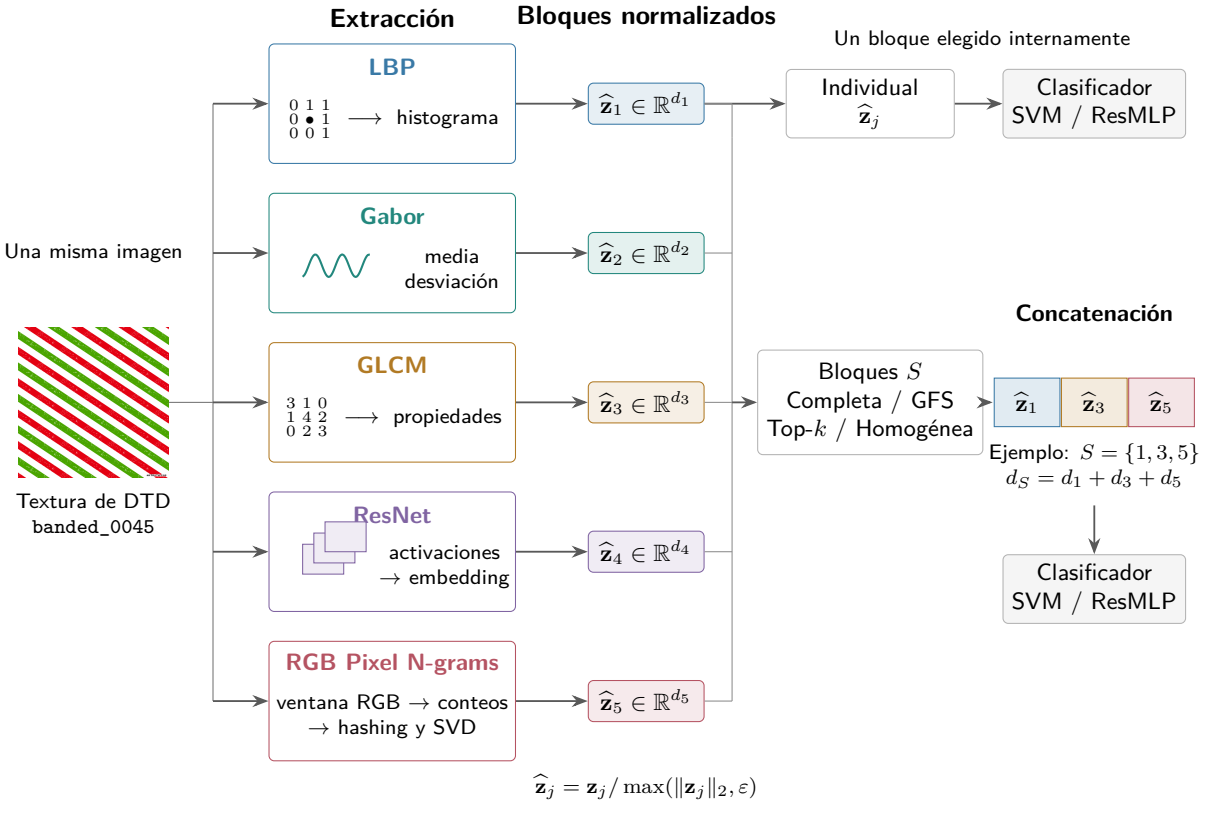


Figura 1: Extracción y concatenación de descriptores. La misma imagen de DTD se representa mediante cinco ejemplos: patrones locales (LBP), respuestas de filtros (Gabor), coocurrencias (GLCM), una red convolucional (ResNet) y secuencias RGB (N-gramas). Las transformaciones internas se muestran esquemáticamente; no representan mapas de importancia por píxel. Cada rama produce un bloque normalizado que puede utilizarse individualmente o concatenarse con otros. En este trabajo, el subconjunto concatenado es la biblioteca completa o una concatenación Top-k.

3.2. Espacio de descriptores

La biblioteca contiene 22 descriptores, agrupados según el tipo de representación. Las Tablas 1 y 2 identifican la parametrización, los checkpoints y la dimensionalidad de cada bloque. Las redes preentrenadas se ejecutaron en modo evaluación y sin ajuste de pesos. En los modelos cargados mediante `timm` se eliminó la capa clasificadora (`num_classes=0`) y se aplicó el preprocesamiento asociado al checkpoint. Para ViT, DeiT, Swin, MAE y SigLIP se utilizó la salida *pooler* cuando estaba disponible y, en caso contrario, el token de clasificación. Todas las imágenes se convirtieron a RGB.

Los descriptores clásicos se basan en operadores LBP, Gabor, GLCM y HOG [4–6, 17, 33]. Las CNN corresponden a VGG16, ResNet, DenseNet, EfficientNet y ConvNeXt V2 [7, 8, 34–36]. El grupo Transformer incluye ViT, DeiT, Swin y EVA-02 [9, 10, 37, 38]; las representaciones con preentrenamiento autosupervisado o multimodal se basan en DINOv2, MAE, SigLIP y BEiTv2 [11, 19, 21, 39].

Los descriptores clásicos se calcularon en escala de grises con la implementación de scikit-image: LBP, LBP-rot, Gabor y GLCM sobre imágenes redimensionadas a 256×256, y HOG sobre 128×128. LBP-rot designa una implementación propia: aplica LBP con mapeo `ror` a ocho rotaciones y concatena diez bins por rotación en el intervalo $[0, 10]$. Los códigos fuera de ese intervalo se descartan; por tanto, es un histograma restringido, no una reproducción del DRLBP de Mehta y Egiazarian [17]. BEiTv2-B utiliza la salida final preclasificador de 768 dimensiones; las imágenes se redimensionan y recortan a 224×224 con la normalización asociada al checkpoint. Los 22 vectores concatenados suman 20.652 dimensiones. Las características fijas y

Tabla 1
Seis descriptores basados en patrones, filtros, gradientes y conteos incluidos en la biblioteca. Cada fila constituye un bloque completo.

Descriptor	Parametrización	Salida	Dim.
LBP	multiescala $(P, R) = (8,1),(16,2),(24,3)$; mapeo uniforme invariante a rotación (riu2)	histogramas normalizados	54
LBP-rot	$P = 8$, $R = 1$; ocho rotaciones; códigos limitados a $[0, 10]$	histogramas 10×8	80
Gabor	cuatro frecuencias 0,1–0,8; seis orientaciones	media y desv. de respuesta real	48
GLCM	256 niveles; distancias 1–3; cuatro ángulos; seis propiedades	promedios sobre ángulos	18
HOG	nueve orientaciones; celda 16^2; bloque 2^2; norma L2-Hys	vector HOG	1.764
N-gramas	ventana vertical 2×1; cuantización RGB por 12; 8.192 bins; SVD	vector normalizado	256

Tabla 2
Dieciséis representaciones aprendidas congeladas incluidas en la biblioteca. "Preclas." designa la salida inmediatamente anterior al clasificador del backbone.

Familia	Descriptor	Checkpoint	Salida	Dim.
CNN	VGG16	`vgg16.tv_in1k`	preclas.	4.096
	ResNet-50	`resnet50.a1_in1k`	preclas.	2.048
	ResNet-101	`resnet101.a1_in1k`	preclas.	2.048
	DenseNet-121	`densenet121.tv_in1k`	preclas.	1.024
	EfficientNet-B0	`efficientnet_b0.ra_in1k`	preclas.	1.280
	ConvNeXt V2-T	`convnextv2_tiny.fcmae_ft_in22k_in1k`	preclas.	768
Transformer	ViT-B/16	`google/vit-base-patch16-224`	pooler/CLS	768
	DeiT-S	`facebook/deit-small-patch16-224`	pooler/CLS	384
	Swin-T	`microsoft/swin-tiny-patch4-window7-224`	pooler	768
	EVA-02 base	`eva02_base_patch14_224`	preclas.	768
Autosup./multimodal	DINOv2-S	`vit_small_patch14_dinov2.lvd142m`	preclas.	384
	DINOv2-B	`vit_base_patch14_dinov2.lvd142m`	preclas.	768
	DINOv2-L	`vit_large_patch14_dinov2.lvd142m`	preclas.	1.024
	MAE-B	`facebook/vit-mae-base`	pooler/CLS	768
	SigLIP-B	`google/siglip-base-patch16-224`	pooler/CLS	768
	BEiTv2-B	`beitv2_base_patch16_224.in1k_ft_in22k_in1k`	preclas.	768

los conteos por imagen se almacenaron junto con etiquetas, orden de muestras y hashes de procedencia; las transformaciones aprendidas se ajustaron dentro de cada partición de entrenamiento.

RGB Pixel N-grams [40] utiliza ventanas verticales de 2×1 píxeles con paso 1 sobre la imagen RGB en su resolución original. Cada canal se cuantiza mediante $q(v) = \lfloor v/12 \rfloor$; los seis valores se codifican en base 22, recorriendo las dos posiciones de R, G y B. Los conteos se agrupan por el resto del código dividido por 8.192 y se comprimen mediante TruncatedSVD aleatorizado (256 componentes, cinco iteraciones y semilla de la condición). SVD se ajusta exclusivamente con el entrenamiento de cada partición interna y se reajusta sobre el entrenamiento externo para la evaluación final. El vector se normaliza con L_2 antes de concatenarlo. Esta parametrización comprimida se identifica como N-gramas + SVD.

3.3. Ranking interno y concatenación Top-k

Para cada condición externa, cada descriptor j se evalúa por separado en cuatro folds internos estratificados y agrupados del entrenamiento externo. Su puntuación es el macro-F1 medio de esa validación,

$r_j = \widehat{F}_{1,\text{inner}}(\{j\})$. Si $\pi_1, \pi_2, \ldots$ ordena los descriptores por puntuación decreciente, la concatenación Top-k es

$$S_k = \{\pi_1, \ldots, \pi_k\}, \tag{2}$$

y su representación $\mathbf{z}_{i,S_k}$ se obtiene con la Ecuación (1). En el uso directo, el clasificador h_{S_k} se entrena con esa representación y, en inferencia, se extraen los k bloques de toda imagen. Top-k no evalúa combinaciones: si dos bloques fuertes son redundantes, ambos pueden quedar entre los primeros.

La curva de k evalúa los prefijos $S_1, \ldots, S_{21}$ de cada condición. En la curva externa, cada prefijo se ajusta con todo el entrenamiento externo y se evalúa una sola vez en el test; en la curva interna, los mismos prefijos se puntúan en los folds internos, sin consultar el test. En ambos barridos los candidatos son los 21 bloques almacenados: N-gramas + SVD, que se reconstruye en cada partición a partir de las imágenes, no participó del ranking. A partir de esas curvas se fija $k = 4$ para todos los datasets y ambos clasificadores (Sección 4.5). El término *Top-4* designa esa concatenación.

3.4. Activación por margen

La activación por margen antepone a Top-k una etapa barata formada por un descriptor b y su clasificador h_b, entrenados con todo el entrenamiento externo. Para una imagen x, sean $s_{(1)}(x) \geq s_{(2)}(x)$ las dos mayores puntuaciones de h_b; en la SVM son valores de la función de decisión uno-contra-el-resto. El margen

$$m(x) = s_{(1)}(x) - s_{(2)}(x) \tag{3}$$

es pequeño cuando la etapa barata duda entre dos clases. La predicción de la cascada es

$$\hat{y}(x) = \begin{cases} h_{S_k}(x), & m(x) \leq \tau_q, \\ h_b(x), & m(x) > \tau_q, \end{cases} \tag{4}$$

de modo que la predicción de Top-k reemplaza a la barata; no se combinan sus puntuaciones. El umbral τ_q es el cuantil q de los márgenes fuera de pliegue (OOF) de h_b sobre el entrenamiento externo, calculados con folds agrupados. Así, q aproxima la fracción de imágenes que se escalan; en el test, la fracción observada ρ_q puede diferir de q.

Si C_b es el costo de la etapa barata y C_j el de extraer el bloque j, el costo medio proyectado por imagen es

$$C(q) = C_b + \rho_q \sum_{j \in S_k \setminus \{b\}} C_j, \tag{5}$$

frente a $C_{\text{dir}} = \sum_{j \in S_k} C_j$ del uso directo; si b pertenece a S_k, su bloque se reutiliza. La cascada ahorra cuando $C(q) < C_{\text{dir}}$, lo que depende tanto de la fracción escalada como de la razón entre los costos. Si Top-k es barata, por ejemplo cuando contiene un solo bloque liviano, la etapa barata puede agregar costo sin ahorrar extracción.

La ruta se elige con datos de entrenamiento (Tabla 3). Las candidatas son Top-k directo y la cascada con cada combinación de etapa barata y cuantil $q \in \{0{,}25;\ 0{,}50;\ 0{,}75\}$. Entre las rutas cuyo macro-F1 OOF no es menor que el de Top-k menos 0,005, se elige la de menor costo proyectado; si ninguna cumple, se usa Top-k directo. Las etapas baratas candidatas son ResNet-50, SigLIP-B, BEiTv2-B y ConvNeXt V2-T; en KTH-TIPS2-b solo ResNet-50 y BEiTv2-B, las únicas con tiempos medidos en ese dataset. Como referencia, la misma regla se aplica restringida a ResNet-50. El protocolo de esta variante se fijó antes de abrir su test; la tolerancia de 0,005, en cambio, proviene de una regla anterior definida después de un análisis exploratorio que había observado resultados externos.

En esta evaluación, la etapa cara es la concatenación Top-k archivada de cada condición. Su k se heredó de la selección secuencial hacia adelante (GFS) de la matriz principal del proyecto, que agrega en cada paso el bloque que más mejora el macro-F1 interno, hasta ocho bloques. Por eso k varía entre 1 y 8 (mediana 5; $k = 4$ en 5 de las 27 condiciones). La comparación entre uso directo y activación por margen es pareada, porque ambas rutas usan el mismo subconjunto, pero no se realizó con $k = 4$ fijo. La Figura 2 resume el flujo.

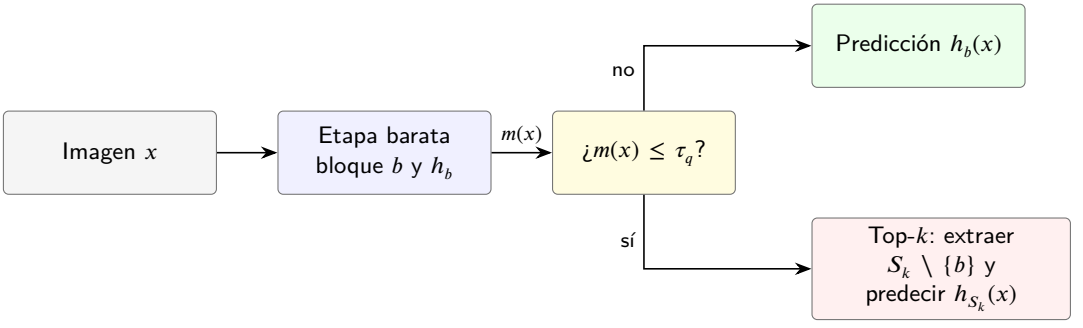


Figura 2: Activación por margen. Toda imagen pasa por la etapa barata; solo las que tienen margen menor o igual que el umbral calculan los bloques de Top-k que faltan, y la predicción de Top-k reemplaza a la barata. El umbral, la etapa barata y la elección entre cascada y Top-k directo se fijan con el entrenamiento externo.

Tabla 3
Configuración de la activación por margen frente a Top-k directo.

Elemento	Configuración
Etapa barata	Un descriptor y su clasificador: ResNet-50, SigLIP-B, BEiTv2-B o ConvNeXt V2-T (en KTH-TIPS2-b, ResNet-50 o BEiTv2-B)
Etapa cara	Concatenación Top-k archivada de la condición, $1 \leq k \leq 8$
Margen	Diferencia entre las dos mayores puntuaciones de la etapa barata
Umbral	Cuantiles 0,25, 0,50 y 0,75 de márgenes OOF agrupados del entrenamiento externo
Compuerta	Se calcula Top-k si $m(x) \leq \tau_q$; su predicción reemplaza a la barata
Elección de ruta	Menor costo proyectado entre las rutas con macro-F1 OOF $\geq$ Top-k $-$ 0,005; si ninguna cumple, Top-k directo
Costo	Perfil de latencias por componente; excluye carga de modelos y ajuste del clasificador
Control	Escalamiento aleatorio de la misma cantidad de imágenes, 100 repeticiones
Información excluida	Test externo y sus etiquetas durante toda la selección

3.5. Variante con rescate por ensamble

Para la medición online de las 94 condiciones se utilizó una variante que conserva la compuerta por margen y cambia la etapa cara. La base se elige en la frontera de Pareto entre macro-F1 OOF y latencia de los 22 descriptores individuales, y el umbral es un cuantil $q \in \{0{,}10; 0{,}25; 0{,}50\}$ de sus márgenes OOF; la compuerta escala si $m(x) < \tau_q$. El rescate agrega las puntuaciones de hasta tres subconjuntos de cuatro bloques tomados de un ensamble por diversidad: 15 subconjuntos muestreados según la relevancia interna de cada bloque, penalizada por su similitud CKA con los demás ($\lambda = 0{,}25$; $\beta = 8$) [41]. La base, el cuantil y los subconjuntos se eligen con tres folds sobre el entrenamiento, maximizando el macro-F1 OOF con un costo medio proyectado no mayor que 60 ms. En lo que sigue esta variante se denomina *política por margen*. No calcula Top-4: sus subconjuntos de rescate tienen cuatro bloques, pero no son necesariamente los cuatro primeros del ranking.

4. Resultados experimentales

4.1. Conjuntos de datos y control de integridad

Antes del análisis se construyó un manifiesto por dataset y se verificaron etiquetas, orden de muestras, dimensiones, valores finitos y grupos de dependencia. Se utilizaron DTD, FMD, CUReT, Outex, Soil y KTH-TIPS2-b.

DTD contiene 5.640 imágenes de texturas en condiciones no controladas y proporciona diez particiones oficiales de entrenamiento, validación y prueba [1]. En cada partición se unieron entrenamiento y validación

para ejecutar la selección interna, mientras que el test oficial se reservó para una única evaluación. Los duplicados byte-idénticos que cruzaban roles fueron retirados del conjunto de entrenamiento, conservando intacta la evaluación.

FMD reúne 1.000 fotografías de diez categorías de materiales [42]. Las características fijas y los conteos por imagen se obtuvieron desde las imágenes oficiales en un orden canónico. Cada imagen presentó un hash único y se utilizó como unidad de agrupación. La evaluación externa empleó cinco folds estratificados y agrupados, repetidos con semillas 42, 123 y 2026.

Outex contiene 1.360 BMP RGB de 128×128 píxeles, organizados en 68 clases balanceadas. La auditoría verificó archivos legibles, IDs continuos 000000–001359, ausencia de duplicados y la partición oficial de 680 imágenes de entrenamiento y 680 de prueba, con diez muestras de cada clase en cada lado [43]. Todos los bloques respetan el orden de las 1.360 muestras del manifiesto oficial.

Para CUReT se utilizó la distribución gris recortada de 61 materiales, con 92 condiciones compartidas por clase [44]. Cada condición constituye un grupo para impedir que la misma configuración de adquisición aparezca a ambos lados de una partición. Se definieron dos mitades complementarias deterministas de 46 condiciones: A para selección y entrenamiento con B como test, y la dirección inversa. Esta asignación reproduce de manera transparente el tamaño 46/46 publicado, pero no se presenta como el split histórico exacto. Debido a que las dos direcciones son complementarias y dependientes, sus resultados se interpretan descriptivamente.

Soil se recuperó de la versión 1 archivada del repositorio *Comprehensive Soil Classification Datasets* de ai4a-lab. Se conservaron sólo las imágenes originales, se eliminaron 49 duplicados exactos y se definieron grupos conservadores a partir de las familias de nombres. El manifiesto final contiene 1.140 imágenes de siete clases y 962 grupos. La evaluación usa cinco folds estratificados y agrupados, repetidos con las semillas 42, 123 y 2026.

KTH-TIPS2-b contiene 4.752 imágenes RGB de 200×200 píxeles, correspondientes a 11 materiales y cuatro muestras físicas por material [45]. Se emplea el protocolo de cuatro particiones usado por RADAM: en cada una, tres muestras físicas por clase quedan en entrenamiento y la restante en prueba. El manifiesto conserva el hash de cada archivo y usa la muestra física como grupo, por lo que ninguna muestra aparece en ambos lados de una partición.

4.2. Clasificadores

Se emplearon dos clasificadores con sesgos diferentes. La SVM lineal se implementa con `LinearSVC` de scikit-learn: $C = 1$, ponderación balanceada por clase, pérdida *squared hinge*, regularización L_2, esquema uno-contra-el-resto, tolerancia 10^{-4} y un máximo de 10.000 iteraciones.

Se denomina ResMLP a la MLP residual implementada para este estudio sobre vectores de características; no es una reproducción del modelo de imágenes del mismo nombre [46]. Proyecta la entrada a 256 unidades y aplica tres bloques residuales, normalización de capa, activaciones GELU, *dropout* de 0,1 y una salida multiclase. Se entrena con entropía cruzada y AdamW, tasa de aprendizaje 10^{-3}, decaimiento de pesos 10^{-4}, lotes de 256 muestras y un máximo de 100 épocas. La parada se basa en la pérdida media de entrenamiento: tras diez épocas sin una disminución mayor que 10^{-4}, se detiene y restaura el mejor estado registrado. No utiliza el test ni una validación adicional para elegir la época. Las semillas se fijan para NumPy y PyTorch; cuando está disponible, la red se ejecuta en GPU.

4.3. Configuración experimental y protocolo anidado

La selección interna utiliza cuatro folds estratificados y agrupados. En DTD se ejecuta dentro de la unión *train+val* de cada split oficial; en FMD y Soil, dentro del entrenamiento de cada fold externo; en CUReT, dentro de la mitad de 46 condiciones asignada al entrenamiento; y en Outex y KTH-TIPS2-b, dentro del entrenamiento definido por su partición oficial. Después de fijar el ranking, el umbral y la ruta, cada modelo se ajusta con todo el entrenamiento externo y se evalúa una sola vez en el test correspondiente. Se verifica programáticamente que ningún grupo aparezca en entrenamiento y prueba.

Los resultados provienen de tres evaluaciones que no se mezclan entre sí. La evaluación estática usa las 47 condiciones externas de cada clasificador: diez de DTD, quince de FMD y quince de Soil, dos de CUReT, una de Outex y cuatro de KTH-TIPS2-b. Sobre ellas se trazan las curvas de k y se compara Top-4 con Individual, el descriptor con mayor macro-F1 interno, y con Completa-22, la concatenación de los 22

bloques. Individual y Completa-22 provienen de la matriz principal del proyecto; Top-4 se ejecutó después con el mismo protocolo y en otro equipo, lo que con ResMLP introduce una variación propia del dispositivo.

La evaluación de la activación por margen usa 27 condiciones con SVM y semilla 42 (DTD 10, FMD 5, CUReT 2, Outex 1, Soil 5 y KTH-TIPS2-b 4) y embeddings almacenados. Sus costos son proyecciones con perfiles de latencia por componente, medidos en KTH-TIPS2-b para ese dataset y en imágenes de entrenamiento de Outex para los demás. La tercera evaluación es un panel online de 94 condiciones, las 47 particiones con ambos clasificadores, en el que cada imagen se procesa individualmente (`batch=1`) en una GPU RTX 4060 y el macro-F1 y la latencia provienen de las mismas llamadas: 470 filas condición–método y 362.640 inferencias. El panel normaliza cada imagen con estadísticas congeladas del entrenamiento, por lo que sus macro-F1 pueden diferir de los de la evaluación estática.

4.4. Análisis estadístico

La exactitud es la proporción de imágenes clasificadas correctamente. Macro-F1 promedia el F1 de las C clases con igual peso: $\text{macro-F1} = C^{-1} \sum_{c=1}^{C} 2TP_c/(2TP_c + FP_c + FN_c)$, donde TP_c, FP_c y FN_c son los verdaderos positivos, falsos positivos y falsos negativos de la clase c. Esta media evita que la contribución de una clase dependa directamente de su número de muestras.

Macro-F1 es la métrica primaria. Para cada contraste se promedian las condiciones externas dentro de cada dataset y clasificador, manteniendo SVM y ResMLP separados. Así, cada prueba usa seis bloques emparejados, uno por dataset, y no trata las particiones repetidas como problemas independientes. La media y la desviación entre particiones describen la variación observada, pero no convierten las particiones dependientes en réplicas independientes.

En la evaluación estática se ejecutan CONTROLTEST y MULTIPLETEST de SCI2S por separado para SVM y ResMLP, siguiendo la presentación del trabajo sobre RGB Pixel N-grams [40] y la metodología de comparaciones múltiples entre datasets [47–49]. La familia contiene ocho estrategias por clasificador: Individual, Completa-22, Top-k con k de GFS, GFS, la selección Homogénea restringida a una familia de descriptores, Top-4 y dos ensambles que promedian 15 subconjuntos de cuatro bloques. Se reportan Friedman, Iman–Davenport, Friedman alineado y Quade, y los valores p de los 28 pares basados en Friedman se ajustan con Holm. La comparación de Top-4 con Completa-22 se plantea además como no inferioridad con un margen $\delta = 0{,}01$ de macro-F1, fijado antes de calcular las pruebas: se informa la cota inferior unilateral al 95 % de la diferencia media (t de Student) y Wilcoxon exacto unilateral.

En la activación por margen, KTH-TIPS2-b se excluye de las pruebas porque su cascada conserva una variante archivada que no se reconcilió con el control Top-k corregido; por tanto, $N = 5$. Se aplican Friedman e Iman–Davenport a cuatro métodos, la cascada con etapa barata elegida, la cascada con ResNet-50, Top-k directo y Completa-22, y Wilcoxon exacto bilateral con Holm a las tres comparaciones de la cascada. El control aleatorio se resume por condición y mediante Wilcoxon exacto bilateral sobre las seis medias por dataset. Se adopta $\alpha = 0{,}05$. La ausencia de significancia no demuestra equivalencia, y las latencias se interpretan descriptivamente.

4.5. Curva de k y elección de $k = 4$

La Figura 3 muestra el macro-F1 medio de los prefijos del ranking interno. Con SVM, la curva externa sube de 0,9174 con un bloque a 0,9366 con cuatro y después oscila entre 0,9348 y 0,9368 hasta los 21 bloques. Con ResMLP, el máximo es 0,9352 con cinco bloques y Top-4 obtiene 0,9331, el 99,8 % de ese máximo. La curva interna de SVM, calculada solo con el entrenamiento, crece 0,0014 de $k = 3$ a $k = 4$ y 0,0009 de $k = 4$ a $k = 5$, alcanza su máximo en $k = 6$ y desciende desde allí. Top-4 ocupa en promedio 3.144 dimensiones con SVM y 3.170 con ResMLP, cerca del 15 % de las 20.652 de la biblioteca.

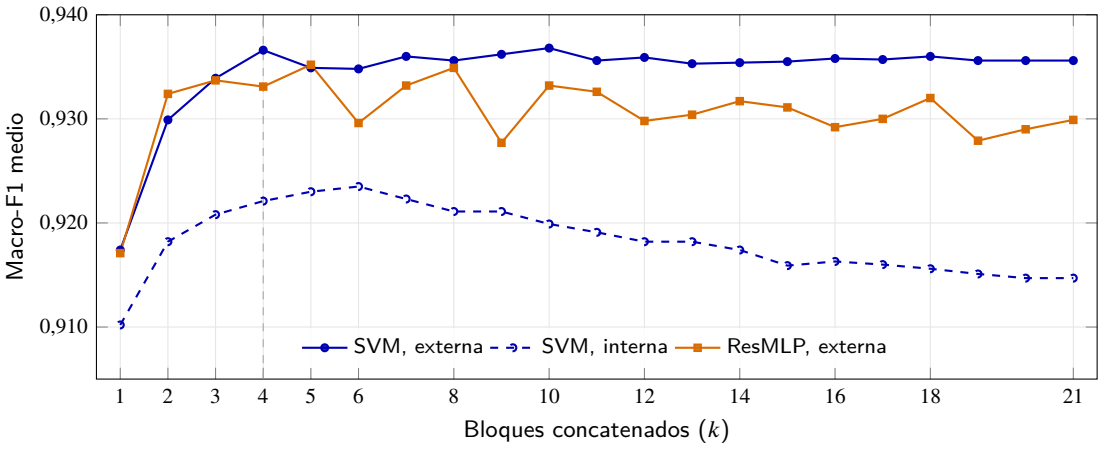


Figura 3: Macro-F1 medio por dataset de los prefijos del ranking interno, en las 47 condiciones de cada clasificador. La curva externa evalúa cada prefijo en el test; la interna lo puntúa en los folds internos del entrenamiento. La línea vertical marca $k = 4$. Los 21 bloques no incluyen N-gramas + SVD.

El valor $k = 4$ se fijó al observar la curva externa de SVM, de modo que esa elección no es independiente del test. La curva interna muestra la misma forma: la regla del codo con tolerancia 0,001, que devuelve el último k antes de la primera ganancia interna menor que la tolerancia, elige $k = 4$ cuando se aplica a la curva interna media de SVM y $k = 6$ con la de ResMLP. Aplicadas condición por condición, en cambio, las reglas internas no recuperan $k = 4$ (Tabla 4). El máximo interno elige en promedio 6,3 bloques con SVM y 7,3 con ResMLP, y el codo con tolerancia 0,001 elige 3,2 y 2,1. Con SVM, ambas reglas quedan por debajo de $k = 4$ fijo (0,9326 y 0,9315 frente a 0,9366); con ResMLP, el máximo interno obtiene 0,9343, por encima de Top-4. La correlación de Spearman entre la curva interna y la externa es positiva pero débil, 0,269 con SVM y 0,321 con ResMLP, y negativa en algunos datasets. Por eso se adopta una constante común en lugar de elegir k por condición.

Tabla 4
Reglas para elegir k con datos de entrenamiento: k medio elegido y macro-F1 externo resultante, con medias por dataset sobre 47 condiciones por clasificador.

	SVM		ResMLP	
Regla	k medio	Macro-F1	k medio	Macro-F1
Máximo de la curva interna	6,3	0,9326	7,3	**0,9343**
Codo, tolerancia 0,001	3,2	0,9315	2,1	0,9295
Codo, tolerancia 0,005	1,9	0,9287	1,7	0,9282
Codo, tolerancia 0,01	1,7	0,9261	1,3	0,9216
Fijo, $k = 3$	3	0,9339	3	0,9337
Fijo, $k = 4$	4	**0,9366**	4	0,9331
Fijo, $k = 6$	6	0,9348	6	0,9296

4.6. Top-4 directo frente a Individual y la concatenación completa

La Tabla 5 compara Top-4 con Individual y Completa-22 en las mismas condiciones. Top-4 supera a Individual en los seis datasets con SVM y en cinco con ResMLP, con ganancias medias de 0,0176 y 0,0147. Frente a Completa-22, Top-4 pierde 0,0026 con SVM y gana 0,0043 con ResMLP. Con SVM supera a Completa-22 en DTD, FMD y Soil y en 35 de las 47 condiciones; con ResMLP lo hace en FMD, Soil y KTH-TIPS2-b y en 29 condiciones. Outex concentra la mayor pérdida con SVM, −0,0368, y Completa-22

conserva ventaja en KTH-TIPS2-b con ese clasificador. En exactitud, Top-4 obtiene 0,9442 con SVM y 0,9420 con ResMLP, frente a 0,9473 y 0,9388 de Completa-22.

Tabla 5
Macro-F1 externo de Individual, Completa-22 y Top-4. Medias entre condiciones externas: 47 por clasificador; Outex tiene un único split oficial. La última fila indica la dimensión media.

	SVM			ResMLP		
Dataset	Individual	Completa-22	Top-4	Individual	Completa-22	Top-4
DTD	0,8426	0,8639	**0,8689**	0,8373	**0,8620**	0,8612
FMD	0,9726	0,9647	**0,9823**	0,9690	0,9564	**0,9800**
CUReT	0,9948	**0,9989**	0,9986	0,9971	**0,9991**	0,9984
Outex	0,9066	**0,9656**	0,9288	**0,9476**	0,9416	0,9357
Soil	0,8578	0,8816	**0,8864**	0,8533	0,8649	**0,8662**
KTH-TIPS2-b	0,9396	**0,9601**	0,9544	0,9061	0,9487	**0,9570**
Media	0,9190	**0,9391**	0,9366	0,9184	0,9288	**0,9331**
Dimensión media	873	20.652	3.144	656	20.652	3.170

En la familia de ocho estrategias, Friedman rechaza la igualdad con ambos clasificadores (SVM: $p = 0{,}0264$; ResMLP: $p = 0{,}0039$), al igual que Iman–Davenport y Quade, mientras que Friedman alineado no rechaza ($p = 0{,}6155$ y $p = 0{,}6150$). Tras ajustar con Holm los 28 pares, ningún par que incluya a Top-4 alcanza el nivel nominal: Top-4 frente a Individual queda en $p_{\text{Holm}} = 0{,}2285$ con SVM y 0,5783 con ResMLP. La Tabla 6 presenta la no inferioridad frente a Completa-22. Con ResMLP se descarta una pérdida de 0,01 (cota inferior $-0{,}0044$; $p = 0{,}0104$). Con SVM no se descarta: la cota inferior es $-0{,}0178$ y $p = 0{,}1850$, principalmente por la pérdida en Outex.

Tabla 6
No inferioridad de Top-4 frente a Completa-22 sobre las medias de seis datasets, con margen $\delta = 0{,}01$ de macro-F1. Cota: límite inferior unilateral al 95 % de la diferencia media (t de Student). p: hipótesis nula de pérdida mayor o igual que 0,01.

Clasif.	Δ medio	Cota 95 %	p (t)	p (Wilcoxon)
SVM	−0,0026	−0,0178	0,1850	0,1562
ResMLP	0,0043	−0,0044	0,0104	0,0156

4.7. Activación por margen frente a Top-k directo

La Tabla 7 compara, en las 27 condiciones con SVM, Top-k usado siempre con su activación por margen bajo la ruta elegida en el entrenamiento. Con la etapa barata elegida, la cascada obtiene un macro-F1 medio de 0,9345 frente a 0,9341 de Top-k directo, con un costo proyectado de 130,0 frente a 529,9 ms por imagen, una reducción del 75,5 %. La regla eligió SigLIP-B como etapa barata en 20 condiciones, BEiTv2-B en cinco y ResNet-50 en una; en la restante, una partición de KTH-TIPS2-b, eligió Top-k directo. Restringida a ResNet-50, la misma regla obtiene 0,9322 con 278,4 ms, un 47,5 % menos que Top-k.

La cascada con base elegida supera a Top-k directo en FMD, Soil y KTH-TIPS2-b, queda por debajo en CUReT y Outex y coincide en DTD. Puede superarlo porque conserva la predicción barata cuando esta es confiable; Top-k no es un techo para la combinación. Completa-22 obtiene la mayor media, 0,9409, por sus ventajas en Outex, Soil y KTH-TIPS2-b. Sin KTH-TIPS2-b ($N = 5$), Friedman ($p = 0{,}4518$) e Iman–Davenport ($p = 0{,}4922$) no rechazan la igualdad de los cuatro métodos, y las tres comparaciones de Wilcoxon de la cascada quedan en $p_{\text{Holm}} = 1{,}0000$. Con cinco datasets, este no rechazo no establece equivalencia ni no inferioridad.

La Tabla 8 examina si el margen elige bien qué imágenes escalar. Fija la etapa barata en ResNet-50 y, para cada cuantil, compara la cascada con un escalamiento de exactamente la misma cantidad de imágenes elegidas al azar, repetido 100 veces por condición. El margen supera al sorteo en las 27 condiciones y en los tres cuantiles; la ventaja media por dataset es de 0,0438, 0,0423 y 0,0261 de macro-F1. Con seis datasets,

Tabla 7
Top-k directo y activación por margen en 27 condiciones con SVM y semilla 42: macro-F1 externo y costo proyectado en ms por imagen. La ruta, el umbral y la etapa barata se eligen con el entrenamiento. Completa-22 se muestra como referencia sobre las mismas condiciones.

		Top-k directo		Cascada, ResNet-50		Cascada, base elegida		Completa-22
Dataset	Cond.	F1	ms	F1	ms	F1	ms	F1
DTD	10	0,8700	661,8	0,8694	498,7	**0,8700**	172,0	0,8639
FMD	5	0,9829	583,4	0,9789	371,6	**0,9840**	138,7	0,9651
CUReT	2	**0,9989**	655,6	0,9979	156,6	0,9982	153,4	**0,9989**
Outex	1	0,9426	729,5	0,9426	344,9	0,9369	165,9	**0,9656**
Soil	5	0,8701	271,3	0,8663	138,1	0,8734	85,2	**0,8919**
KTH-TIPS2-b	4	0,9400	278,0	0,9381	160,5	0,9445	64,9	**0,9601**
Media	27	0,9341	529,9	0,9322	278,4	0,9345	130,0	**0,9409**

Wilcoxon exacto bilateral alcanza su mínimo posible, $p = 0{,}0312$, en cada cuantil ($p_{\text{Holm}} = 0{,}0938$ al ajustar por los tres). Al escalar el 69 % de las imágenes, la cascada queda a 0,0003 de Top-k siempre y lo iguala o supera en 17 de 27 condiciones; al escalar el 22 %, pierde 0,0229. ResNet-50 sola obtiene 0,8483.

Tabla 8
Cascada con ResNet-50 y cuantil fijo frente a un escalamiento aleatorio de la misma cantidad de imágenes, en 27 condiciones con SVM. Medias por dataset de macro-F1; la fracción observada es la proporción de imágenes de test escaladas. ResNet-50 sola obtiene 0,8483.

Cuantil q	Fracción observada	Cascada	Aleatorio	Top-k siempre	Cascada > aleatorio	Cascada ≥ Top-k
0,25	0,217	0,9112	0,8674	0,9341	27/27	1/27
0,50	0,450	0,9300	0,8877	0,9341	27/27	9/27
0,75	0,687	0,9338	0,9077	0,9341	27/27	17/27

4.8. Medición física de la cascada

Para contrastar la proyección con tiempos reales, se midió una condición por dataset en una GPU RTX 4060, con modelos residentes e imágenes ya decodificadas: una imagen por clase, tres repeticiones pareadas y orden aleatorizado, en total 204 imágenes y 612 pares (Tabla 9). El agregado registra 90,25 ms por imagen para la cascada frente a 291,21 ms de Top-k directo, una reducción del 69,0 %; 453 de las 612 ejecuciones terminaron en la etapa barata. Las rutas y predicciones coincidieron con las calculadas sobre los embeddings almacenados, pero los cosenos mínimos entre las representaciones extraídas y las almacenadas fueron −0,0707 en CUReT y 0,0085 en Soil. Por eso no puede afirmarse que la medición física reproduzca exactamente el sistema almacenado. KTH-TIPS2-b conserva la variante histórica de la cascada.

Tabla 9
Piloto físico de la cascada en RTX 4060: una condición por dataset, tres repeticiones pareadas por imagen. Las repeticiones comparten imagen, por lo que los pares no son observaciones independientes.

Dataset	Imágenes	Pares	Cascada (ms)	Top-k (ms)	Reducción
DTD	47	141	90,57	295,55	69,4 %
FMD	10	30	13,01	208,08	93,7 %
CUReT	61	183	92,52	270,60	65,8 %
Outex	68	204	111,47	354,31	68,5 %
Soil	7	21	41,99	109,10	61,5 %
KTH-TIPS2-b	11	33	46,08	188,33	75,5 %
Agregado	204	612	90,25	291,21	69,0 %

4.9. Panel online: Top-4 directo y política por margen

La Tabla 10 reúne el panel online. En las doce combinaciones de dataset y clasificador, la política por margen tiene la menor latencia media, entre 21,65 y 54,35 ms por imagen, con medianas entre 2,81 y 25,18 ms porque la mayoría de las imágenes termina en la etapa barata. Top-4 tarda entre 70,98 y 333,98 ms, y Completa-22 entre 1.551 y 1.823 ms. La razón de latencias entre Top-4 y la política por margen va de 1,6 en Soil con ResMLP, donde Top-4 ya es barato, a 12,2 en Outex con ResMLP.

Tabla 10
Panel online de 94 condiciones (`batch=1`, RTX 4060): macro-F1 medio entre particiones y latencia media en ms por imagen, medidos en las mismas llamadas. La política por margen rescata con subconjuntos de cuatro bloques de un ensamble por diversidad (Sección 3.5).

		Top-4		Política por margen		Completa-22	
Dataset	Clasif.	F1	ms	F1	ms	F1	ms
DTD[†]	SVM	**0,8689**	333,98	0,8669	54,35	0,8635	1.611,57
	ResMLP	0,8613	267,50	**0,8660**	47,61	0,8616	1.551,34
FMD	SVM	**0,9823**	268,96	0,9776	38,63	0,9651	1.759,50
	ResMLP	**0,9800**	258,56	0,9760	36,35	0,9577	1.759,70
CUReT	SVM	0,9986	270,69	0,9984	36,83	**0,9989**	1.712,57
	ResMLP	0,9984	268,42	0,9989	30,47	**0,9991**	1.618,51
Outex	SVM	0,9288	265,38	0,9276	40,33	**0,9552**	1.574,19
	ResMLP	0,9275	263,77	**0,9609**	21,65	0,9477	1.563,18
Soil	SVM	0,8594	70,98	0,8725	38,48	**0,8792**	1.651,19
	ResMLP	0,8633	78,21	0,8668	48,33	**0,8673**	1.823,25
KTH-TIPS2-b	SVM	0,9544	260,38	0,9390	30,95	**0,9599**	1.579,08
	ResMLP	**0,9523**	260,11	0,9391	32,16	0,9461	1.626,70
Media	SVM	0,9321	245,06	0,9303	39,93	**0,9370**	1.648,02
	ResMLP	0,9305	232,76	**0,9346**	36,10	0,9299	1.657,11

[†]En DTD con SVM se conserva una reserva sobre la medición de tiempos por una pausa durante una ventana nocturna; la mediana de la política por margen es 14,10 ms con SVM y 14,77 ms con ResMLP.

En macro-F1, la media por dataset es 0,9321 para Top-4 y 0,9303 para la política por margen con SVM, y 0,9305 y 0,9346 con ResMLP. Con SVM, la política por margen queda por debajo de Top-4 en cinco de seis datasets, con la mayor pérdida en KTH-TIPS2-b (−0,0154) y una ganancia en Soil (+0,0131). Con ResMLP supera a Top-4 en cuatro datasets, con la mayor ganancia en Outex (+0,0334) y la mayor pérdida en KTH-TIPS2-b (−0,0132). Completa-22 obtiene la mayor media con SVM, 0,9370, con una latencia 41 veces mayor que la de la política por margen. Los macro-F1 del panel no coinciden exactamente con los de la Tabla 5; la mayor diferencia corresponde a Top-4 con SVM en Soil (0,8594 frente a 0,8864).

4.10. Comparación con métodos recientes en el panel común

Los cuatro métodos de la Sección 2 se reimplementaron y evaluaron sobre las mismas particiones del panel, con llamadas individuales (Tabla 11). Antes de compararlos se verificó cada implementación contra el código o los ejemplos numéricos publicados cuando estaban disponibles. En macro-F1, la política por margen con SVM supera a los cuatro métodos en los seis datasets. Frente al mejor de ellos, la agregación multicapa de BEiTv2 de Neshov et al., la ventaja va de 0,0044 en CUReT a 0,0648 en DTD. Neshov et al. es, en cambio, más rápido: tarda entre 14,27 y 19,17 ms por imagen, frente a 30,95–54,35 ms de la política por margen. De las 230 comparaciones por dataset entre las estrategias propias y los métodos externos, la única con diferencia negativa de macro-F1 es Top-4 con SVM en Soil frente a Neshov et al. (−0,0037).

HyTexNet y 6LBP+dCellNN obtienen valores muy inferiores a los publicados. El panel separa las muestras físicas entre entrenamiento y prueba, mientras que sus protocolos originales no lo hacen: HyTexNet alcanza 0,9394 de macro-F1 en el protocolo de un tercio de KTH-TIPS2-b, que comparte muestras físicas, y 0,8259 en el panel. En DeepLumina, 1.653 de los 2.441 clasificadores binarios entrenados alcanzaron el máximo de 10.000 iteraciones, y su efecto sobre el macro-F1 no está cuantificado. Por estas diferencias de protocolo, la Tabla 11 contextualiza magnitudes y no demuestra superioridad.

Tabla 11
Comparación descriptiva en el panel común: macro-F1 medio por dataset y, entre paréntesis, latencia media en ms por imagen (`batch=1`). Los métodos externos se reimplementaron sobre las mismas particiones; las latencias se midieron en fechas distintas y con hardware compartido.

Método	DTD	FMD	CUReT	Outex	Soil	KTH-TIPS2-b
6LBP+dCellNN [32]	0,2162 (31,5)	0,2477 (29,7)	0,6934 (31,0)	0,6356 (29,9)	0,3550 (33,0)	0,5750 (31,9)
HyTexNet [31]	0,6242 (33,5)	0,7414 (31,2)	0,9831 (32,4)	0,8927 (29,0)	0,7880 (33,1)	0,8259 (31,0)
DeepLumina‡ [30]	—	0,8559 (53,3)	0,9881 (53,0)	0,9073 (53,5)	0,8050 (57,8)	0,8744 (51,6)
Neshov et al. [28]	0,8020 (16,8)	0,9421 (16,9)	0,9940 (14,3)	0,9193 (14,6)	0,8631 (19,2)	0,9245 (16,5)
Top-4, SVM	**0,8689** (334,0)	**0,9823** (269,0)	0,9986 (270,7)	0,9288 (265,4)	0,8594 (71,0)	**0,9544** (260,4)
Política por margen, SVM	0,8669 (54,4)	0,9776 (38,6)	0,9984 (36,8)	0,9276 (40,3)	**0,8725** (38,5)	0,9390 (31,0)
Política por margen, ResMLP	0,8660 (47,6)	0,9760 (36,4)	**0,9989** (30,5)	**0,9609** (21,7)	0,8668 (48,3)	0,9391 (32,2)

‡DeepLumina queda sujeto a revisión: en todas sus condiciones al menos un clasificador binario alcanzó el máximo de iteraciones, y en DTD una de las diez condiciones no terminó, por lo que no se informa media.

5. Discusión

Los resultados indican que buena parte del beneficio de concatenar se obtiene con pocos bloques. Con el ranking interno, los cuatro primeros descriptores son casi siempre Transformers preentrenados a gran escala: con SVM, SigLIP-B y EVA-02 aparecen en 41 de los 47 subconjuntos Top-4, DINOv2-B y DINOv2-L en 32 y BEiTv2-B en 19. Agregar más bloques del mismo ranking incorpora representaciones con información en parte redundante, y la curva externa deja de crecer. Top-4 mejora a Individual y queda cerca de la concatenación completa, pero no la reemplaza en todos los dominios: con SVM pierde 0,0368 en Outex y Completa-22 conserva ventaja en KTH-TIPS2-b. N-gramas + SVD, el bloque de color, no participó del ranking y, dentro de la concatenación completa, aporta 0,0078 en Soil y 0,0105 en KTH-TIPS2-b con SVM. Un Top-k limitado a los bloques individualmente más fuertes puede omitir así información complementaria.

La activación por margen conserva el macro-F1 de Top-k directo con una fracción de su costo porque la mayoría de las imágenes se resuelve en la etapa barata y el margen ordena bien la dificultad. El control aleatorio aísla ese efecto: escalar la misma cantidad de imágenes al azar pierde entre 0,026 y 0,044 de macro-F1 frente al margen. La cascada puede incluso superar a Top-k directo, como en FMD, Soil y KTH-TIPS2-b, porque conserva la predicción barata cuando esta es correcta y Top-k se equivoca.

El ahorro depende de la razón entre el costo de la etapa barata y el de Top-k. Cuando Top-k contiene bloques costosos como DINOv2-L, que tardó 476 ms por imagen en una medición sobre KTH-TIPS2-b frente a 37 ms de ResNet-50, la cascada evita la mayor parte de la extracción. Cuando Top-k es liviano, la etapa barata agrega costo: en una medición online de KTH-TIPS2-b con 30 imágenes por partición, la cascada fue más lenta que Top-k directo en las dos particiones cuyo Top-k tenía un solo bloque (49,45 frente a 42,61 ms y 57,17 frente a 40,07 ms). En el panel, la razón de latencias frente a Top-4 cae a 1,6–1,8 en Soil. La regla de costo de la Sección 3.4 permite elegir Top-k directo en esos casos antes de ver el test, siempre que el perfil de latencias represente el equipo de uso.

El panel muestra que la compuerta por margen también reduce la latencia cuando el rescate es un ensamble de subconjuntos de cuatro bloques: la política por margen es más rápida que Top-4 en las doce combinaciones, con un macro-F1 medio similar. Sus pérdidas frente a Top-4 se concentran en KTH-TIPS2-b con ambos clasificadores. Como su etapa cara no es Top-4, el panel no reemplaza a la comparación pareada de la Sección 4.7; la complementa con latencias medidas en las 94 condiciones.

5.1. Limitaciones

El valor $k = 4$ se leyó de la curva externa de SVM. La curva interna media muestra la misma forma, pero las reglas aplicadas por condición no lo recuperan, de modo que la elección no es independiente del test. Ese valor se aplicó sin cambios a ResMLP, cuya curva externa alcanza su máximo con cinco bloques, y el ranking no incluyó N-gramas + SVD.

La comparación pareada entre uso directo y activación por margen utiliza la Top-k archivada de cada condición, con k entre 1 y 8, y no Top-4; la medición con Top-4 directo proviene del panel, cuya etapa cara es un ensamble. Esa comparación usa solo SVM y una semilla. La tolerancia de 0,005 se fijó tras un

análisis exploratorio con resultados externos, y la ruta Top-k no se reconstruyó dentro de cada fold productor de márgenes OOF, lo que puede favorecer su estimación interna. KTH-TIPS2-b conserva una variante no reconciliada, por lo que las pruebas de la cascada usan cinco datasets.

Los costos de la cascada son proyecciones por componente y no incluyen carga de modelos ni ajuste del clasificador. El piloto físico mide una condición por dataset y presenta discrepancias de representación en CUReT y Soil. Las latencias del panel se midieron en fechas distintas y con hardware compartido, y en DTD con SVM se registró una pausa durante una ventana nocturna. Las comparaciones con métodos publicados son descriptivas: se reimplementaron en un protocolo común que difiere de los originales, y DeepLumina presenta problemas de convergencia.

Con seis datasets, las pruebas tienen poca potencia y no se realizó un cálculo de tamaño muestral a priori. Las particiones de DTD y FMD reutilizan imágenes, CUReT presenta dos direcciones complementarias y Outex tiene una sola partición. Los descriptores clásicos se calcularon en escala de grises, mientras las redes y N-gramas + SVD utilizan RGB, y el posible solapamiento entre los corpus de preentrenamiento y los benchmarks no fue auditado.

6. Conclusión

Este trabajo comparó dos usos de una concatenación Top-k de descriptores para clasificar texturas: calcularla para toda imagen o activarla solo cuando una etapa barata duda. Con 22 descriptores, seis datasets y dos clasificadores, la curva de desempeño se aplana alrededor de cuatro bloques. Top-4 ocupa cerca del 15 % de las dimensiones de la biblioteca, supera al descriptor individual elegido internamente en once de doce combinaciones de dataset y clasificador, y queda a 0,0026 de la concatenación completa con SVM, mientras la supera en 0,0043 con ResMLP. La no inferioridad con margen 0,01 solo se establece con ResMLP.

La activación por margen conservó el macro-F1 de la misma Top-k usada siempre (0,9345 frente a 0,9341) con un costo proyectado 75,5 % menor, y un piloto en GPU midió una reducción del 69 %. El margen eligió qué imágenes escalar mejor que un sorteo con la misma cantidad en todas las condiciones evaluadas. En el panel online, la política por margen con rescate de cuatro bloques procesó cada imagen en 22 a 54 ms, frente a 71–334 ms de Top-4, con un macro-F1 medio similar. Las pruebas no detectaron diferencias entre los usos comparados, lo que no demuestra equivalencia.

En la práctica, Top-4 es una alternativa compacta a la concatenación completa, y su activación por margen conviene cuando la etapa barata es mucho más rápida que los bloques de Top-k. Como siguientes pasos se plantea repetir la comparación pareada con Top-4 fijo y ambos clasificadores, incluir el bloque de color en el ranking y medir la latencia de forma homogénea en un mismo equipo.

A. Trazabilidad de resultados

Las curvas de k y los resultados de Top-4 provienen de `results/exploratory/k_sweep/` (SVM) y `results/exploratory/resmlp/k_sweep/` (ResMLP), en los archivos `k_sweep.csv` y `k_sweep_inner.csv`; las reglas para elegir k, de `results/analysis/k_selection_rules/`; y las pruebas de la evaluación estática, de `results/analysis/selection_stats/`. Individual y Completa-22 proceden de la matriz principal de 94 condiciones del proyecto.

La activación por margen se reconstruye desde `results/exploratory/topk_cascade/`: los directorios `*/exact_archive/` contienen los 27 archivos por condición con cuantiles fijos y control aleatorio; `cheap_stage_selected_primary22/summary.json`, la ruta con etapa barata elegida; y `train_selected_primary22/selected_summary.json`, la ruta con ResNet-50. El piloto físico está en `results/exploratory/fsc_tex/cascade_physical_paired_benchmark.json`. El panel online y la comparación con métodos externos están en `results/analysis/reconciliation_full_panel_20261104_v1/`, que reconstruye las 230 comparaciones por dataset desde las llamadas guardadas. Los archivos por condición registran hashes SHA-256 de manifiestos, embeddings y ejecutores.

Disponibilidad de datos y código

Los datasets utilizados son de acceso público y permanecen sujetos a las condiciones de sus respectivos distribuidores. El repositorio del proyecto contiene el código, los manifiestos, las salidas por partición, los resultados estadísticos y los hashes necesarios para auditar las tablas; los datasets originales y los embeddings se obtienen o regeneran según sus respectivas condiciones de uso. El material público está versionado en Git, aunque este borrador todavía no identifica un depósito archivado con DOI o una versión inmutable independiente del repositorio.

Declaración ética

Los experimentos utilizan colecciones públicas de imágenes de texturas y no incorporan reclutamiento de participantes. La declaración ética final queda pendiente de confirmación por los autores.

CRediT authorship contribution statement

Carlos Ayala: Conceptualización, Metodología, Software, Experimentación, Análisis formal, Redacción – borrador original. **Joaquín Delgado:** Conceptualización, Metodología, Software, Experimentación, Análisis formal, Redacción – revisión y edición.

Referencias

[1] Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3606–3613, 06 2014. doi: 10.1109/CVPR.2014.461.

[2] Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, and Andrea Vedaldi. Deep filter banks for texture recognition, description, and segmentation. *International Journal of Computer Vision (IJCV)*, 118(1):65–94, 01 2016. doi: 10.1007/s11263-015-0872-3.

[3] Li Liu, Jie Chen, Paul Fieguth, Guoying Zhao, Rama Chellappa, and Matti Pietik"ainen. From BoW to CNN: Two decades of texture representation for texture classification. *International Journal of Computer Vision (IJCV)*, 127:74–109, 2019. doi: 10.1007/s11263-018-1125-z.

[4] Timo Ojala, Matti Pietikäinen, and Topi Mäenpää. Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 24(7):971–987, 07 2002. doi: 10.1109/TPAMI.2002.1017623.

[5] Robert M. Haralick. Statistical and structural approaches to texture. *Proceedings of the IEEE*, 67(5):786–804, 05 1979. doi: 10.1109/PROC.1979.11328.

[6] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 886–893. IEEE, 06 2005. doi: 10.1109/CVPR.2005.177.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 06 2016. doi: 10.1109/CVPR.2016.90.

[8] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114. PMLR, 06 2019. URL https://proceedings.mlr.press/v97/tan19a.html.

[9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 05 2021. URL https://openreview.net/forum?id=YicbFdNTTy. International Conference on Learning Representations (ICLR).

[10] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10012–10022, 10 2021. doi: 10.1109/ICCV48922.2021.00986.

[11] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 07 2024. URL https://arxiv.org/abs/2304.07193.

[12] Jorge Sánchez, Florent Perronnin, Thomas Mensink, and Jakob Verbeek. Image classification with the fisher vector: Theory and practice. *International Journal of Computer Vision (IJCV)*, 105(3):222–245, 12 2013. doi: 10.1007/s11263-013-0636-x.

[13] Hang Zhang, Jia Xue, and Kristin Dana. Deep TEN: Texture encoding network. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2896–2905. IEEE, 07 2017. doi: 10.1109/CVPR.2017.309.

[14] Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages I–511. IEEE, 2001. doi: 10.1109/CVPR.2001.990517.

[15] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. Branchynet: Fast inference via early exiting from deep neural networks. In *23rd International Conference on Pattern Recognition (ICPR)*, pages 2464–2469. IEEE, 2016. doi: 10.1109/ICPR.2016.7900006.

[16] Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. Dynamic neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 44(11):7436–7456, 2021. doi: 10.1109/TPAMI.2021.3117837.

[17] Rakesh Mehta and Karen Egiazarian. Dominant rotated local binary patterns (DRLBP) for texture classification. *Pattern Recognition Letters*, 71:16–22, 02 2016. doi: 10.1016/j.patrec.2015.11.019.

[18] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9650–9660, 10 2021. doi: 10.1109/ICCV48922.2021.00951.

[19] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009. IEEE, 06 2022. doi: 10.1109/CVPR52688.2022.01553.

[20] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 07 2021. URL https://proceedings.mlr.press/v139/radford21a.html.

[21] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11941–11952. IEEE, 10 2023. doi: 10.1109/ICCV51070.2023.01100.

[22] Ron Kohavi and George H. John. Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2):273–324, 1997. doi: 10.1016/S0004-3702(97)00043-X.

[23] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003. URL https://www.jmlr.org/papers/v3/guyon03a.html.

[24] Gao Huang, Danlu Chen, Tianhong Li, Felix Wu, Laurens van der Maaten, and Kilian Q. Weinberger. Multi-scale dense networks for resource-constrained image classification. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://openreview.net/forum?id=Hk2myTqex.

[25] Zhixiang Xu, Kilian Q. Weinberger, and Olivier Chapelle. The greedy miser: Learning under test-time budgets. In *Proceedings of the 29th International Conference on Machine Learning*, 2012. URL https://icml.cc/2012/papers/592.pdf.

[26] Feng Nan and Venkatesh Saligrama. Adaptive classification for prediction under a budget. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/d9ff90f4000eacd3a6c9cb27f78994cf-Abstract.html.

[27] Chao Ma, Sebastian Tschiatschek, Konstantina Palla, José Miguel Hernández-Lobato, Sebastian Nowozin, and Cheng Zhang. EDDI: Efficient dynamic discovery of high-value information with partial VAE. In *International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 4234–4243. PMLR, 2019. URL https://proceedings.mlr.press/v97/ma19c.html.

[28] Nikolay Neshov, Krasimir Tonchev, Ivaylo Bozhilov, Radostina Petkova, and Agata Manolova. Improving texture recognition via multi-layer feature aggregation from pre-trained vision architectures. *Electronics*, 14(23):4779, 2025. doi:

10.3390/electronics14234779.

[29] Leonardo Scabini, Arthur Sacilotti, Kamil M. Zielinski, Lucas C. Ribas, Bernard De Baets, and Odemir M. Bruno. A comparative survey of vision transformers for feature extraction in texture analysis. *Journal of Imaging*, 11(9):304, 2025. doi: 10.3390/jimaging11090304.

[30] P. Simon and V. Uma. DeepLumina: A method based on deep features and luminance information for color texture classification. *Computational Intelligence and Neuroscience*, 2022:9510987, 2022. doi: 10.1155/2022/9510987.

[31] Vandana Gupta, Ashish Mishra, and Nishant Shrivastava. HyTexNet: Percentile-guided local encoding and deep feature fusion for enhanced texture classification. *Knowledge-Based Systems*, 338:115482, 2026. doi: 10.1016/j.knosys.2026.115482.

[32] Mingzhe Chang, Luping Ji, and Jiewen Zhu. Multi-scale LBP fusion with the contours from deep CellNNs for texture classification. *Expert Systems with Applications*, 238:122100, 2024. doi: 10.1016/j.eswa.2023.122100.

[33] Anil K. Jain and Farshid Farrokhnia. Unsupervised texture segmentation using Gabor filters. *Pattern Recognition*, 24 (12):1167–1186, 1991. doi: 10.1016/0031-3203(91)90143-S.

[34] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2015. URL https://arxiv.org/abs/1409.1556. International Conference on Learning Representations (ICLR).

[35] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2261–2269. IEEE, 07 2017. doi: 10.1109/CVPR.2017.243. URL https://openaccess.thecvf.com/content_cvpr_2017/html/Huang_Densely_Connected_Convolutional_CVPR_2017_paper.html.

[36] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. Convnext v2: Co-designing and scaling convnets with masked autoencoders. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16133–16142, 06 2023. doi: 10.1109/CVPR52729.2023.01548.

[37] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 10347–10357. PMLR, 07 2021. URL https://proceedings.mlr.press/v139/touvron21a.html.

[38] Yuxin Fang, Quan Sun, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. Eva-02: A visual representation for neon genesis. *Image and Vision Computing*, 149:105171, 09 2024. doi: 10.1016/j.imavis.2024.105171.

[39] Zhiliang Peng, Li Dong, Hangbo Bao, Qixiang Ye, and Furu Wei. Beit v2: Masked image modeling with vector-quantized visual tokenizers. *arXiv preprint arXiv:2208.06366*, 2022. doi: 10.48550/arXiv.2208.06366.

[40] Fátima Belén Paiva Pavón, María Cristina Orué Gil, José Luis Vázquez Noguera, Helena Gómez-Adorno, and Valentín Calzada-Ledesma. Rgb pixel n-grams: A texture descriptor. *Signal Processing: Image Communication*, 118:117028, 2023. doi: 10.1016/j.image.2023.117028.

[41] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3519–3529, 2019. URL https://proceedings.mlr.press/v97/kornblith19a.html.

[42] Lavanya Sharan, Ce Liu, Ruth Rosenholtz, and Edward H. Adelson. Recognizing materials using perceptually inspired features. *International Journal of Computer Vision (IJCV)*, 103(3):348–371, 07 2013. doi: 10.1007/s11263-013-0609-0.

[43] Timo Ojala, Topi Mäenpää, Matti Pietikäinen, Jaakko Viertola, Juha Kyllönen, and Sami Huovinen. Outex—new framework for empirical evaluation of texture analysis algorithms. In *Proceedings of the 16th International Conference on Pattern Recognition*, volume 1, pages 701–706. IEEE, 2002. doi: 10.1109/ICPR.2002.1044854.

[44] Manik Varma and Andrew Zisserman. Texture classification: Are filter banks necessary? In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 2, pages 691–698. IEEE, 2003. URL https://www.robots.ox.ac.uk/~vgg/publications/2003/Varma03/varma03.pdf.

[45] P. Mallikarjuna, Mario Fritz, Eric Hayman, Barbara Caputo, and Jan-Olof Eklundh. Kth-tips and kth-tips2: Two databases for the evaluation of texture classification algorithms. In *Swedish Symposium on Image Analysis*, 03 2005.

[46] Hugo Touvron, Piotr Bojanowski, Mathilde Caron, Matthieu Cord, Alaaeldin El-Nouby, Edouard Grave, Gautier Izacard, Armand Joulin, Gabriel Synnaeve, Jakob Verbeek, and Hervé Jégou. ResMLP: Feedforward networks for image classification with data-efficient training. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 45(4):5314–5321, 04 2023. doi: 10.1109/TPAMI.2022.3206148.

[47] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7: 1–30, 2006.

[48] Salvador García and Francisco Herrera. An extension on "statistical comparisons of classifiers over multiple data sets" for all pairwise comparisons. *Journal of Machine Learning Research (JMLR)*, 9(89):2677–2694, 2008. URL https://jmlr.org/papers/v9/garcia08a.html.

[49] Joaquín Derrac, Salvador García, Daniel Molina, and Francisco Herrera. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation*, 1(1):3–18, 2011. doi: 10.1016/j.swevo.2011.02.002.